

Svelte - Komponenten

HINWEIS: Diese Unterlagen basieren auf den Skripten von Prof. Grüneis.

Komponenten sind die Bausteine von Svelte-Anwendungen. Sie ermöglichen es, wiederverwendbare und modulare UI-Elemente zu erstellen. Jede Komponente besteht aus drei Hauptteilen: dem Skript, dem Markup und den Styles.

Beispiel: Bücherliste

Angenommen, wir möchten eine App zum Verwalten von Büchern entwickeln. Auf der Startseite möchten wir eine Liste von Büchern anzeigen. Die Bücher sind in der Datei **books.json** gespeichert. Das File hat folgende Struktur:

```
[
  {
    "id": 1,
    "title": "The Great Gatsby",
    "author": "F. Scott Fitzgerald",
    "year": "1925"
  },
  ...
]
```

Die Anzeige soll so aussehen:

Books				
Anzahl Bücher: 5				
1	The Great Gatsby	F. Scott Fitzgerald	1925	
2	To Kill a Mockingbird	Harper Lee	1960	
3	1984	George Orwell	1949	
4	Animal Farm	George Orwell	1945	
5	Brave New World	Aldous Huxley	1932	

Die `+page.svelte`-Datei könnte wie folgt aussehen:

```
<script>
  import books from '$lib/data/books.json';

  let bookList = $state.raw(books);

  function deleteBook(book) {
    console.log(`deleteBook(...) - book.id = ${book.id}`);
    bookList = bookList.filter((x) => x.id !== book.id);
  }
</script>

<h1>Books</h1>
<hr />
Anzahl Bücher: {bookList.length}
<div class="table">
  {#each bookList as book}
    <div class="row">
      <div class="cell id">{book.id}</div>
      <div class="cell title">{book.title}</div>
      <div class="cell author">{book.author}</div>
      <div class="cell year">{book.year}</div>
      <div class="cell actions">
        <button onclick={() => deleteBook(book)} class="mdl2 warn">
          <i class="bi bi-trash"></i>
        </button>
      </div>
    </div>
  {/each}
</div>
```

```
<style lang="scss">
  <!-- siehe style.scss -->
</style>
```

In **+layout.svelte** müssen wir noch folgendes ergänzen (für die Bootstrap-Icons):

```
<svelte:head>
  <link rel="icon" href={favicon} />
  <link
    rel="stylesheet"
    ↪ href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.13.1/font/bootstrap-ico
    />
</svelte:head>
```

\$state() vs. \$state.raw()

`$state()` erzeugt "deeply reactive" State, `$state.raw()` liefert dagegen ein nicht tief reaktives ("rohes") Objekt bzw. Array.

Was macht \$state()?

Erstellt eine reaktive Variable, deren Wert komplett "deep proxied" wird.

Wenn du in einem Objekt- oder Array-State irgendetwas änderst (Property setzen, push, splice, verschachtelte Objekte verändern usw.), wird automatisch ein Re-Render ausgelöst.

Beispiel:

```
let person = $state({ name: 'Ada', address: { city: 'Linz' } });

function changeCity() {
  person.address.city = 'Graz'; // löst Update aus
}
```

Was macht `$state.raw()`?

Erstellt State, der nicht tief reaktiv gemacht wird, sondern als "plain value" behandelt wird.

Das Objekt/Array selbst wird nicht mit einem Proxy umwickelt; Svelte beobachtet nur Zuweisungen an die Variable, nicht interne Mutationen.

Wichtig: Du sollst den Inhalt nicht mutieren, sondern immer ein neues Objekt/Array zuweisen.

Beispiel:

```
let person = $state.raw({ name: 'Ada', address: { city: 'Linz' } });
```

```
function changeCityWrong() {  
  person.address.city = 'Graz'; // keine Reaktivität, kein Re-Render  
}
```

```
function changeCityRight() {  
  person = { ...person, address: { ...person.address, city: 'Graz' }  
    ↪ };  
  // neue Referenz -> Re-Render  
}
```

Praktischer Unterschied:

- **`$state()`:**

- Deep reactive.
- Du kannst "mutativ" arbeiten (z.B. `arr.push(...)`, `obj.foo = 1`), und die UI bleibt synchron.
- Gut, wenn du State intensiv und tief verschachtelt änderst und dir Bequemlichkeit wichtiger ist als Feintuning bei Performance.

- **`$state.raw()`:**

- Nicht tief reaktiv, Referenzgleichheit bleibt vollständig unter deiner Kontrolle.
- Reaktivität nur bei Reassignment (`foo = neuerWert`), nicht bei inneren Mutationen.
- Gut, wenn du:
 - * bewusst immutable Patterns nutzen möchtest (immer neue Objekte erzeugen),
 - * Performance optimieren willst (keine tiefen Proxies),
 - * oder du externe/Third-Party-Objekte speicherst, die du nicht von Svelte "anfassen" lassen willst.

SCSS aktivieren

Um mit SCSS arbeiten zu können, brauchen wir noch ein Node-Modul für **scss**:

```
npm install -D sass-embedded
```

Und zuletzt müssen wir in der **svelte.config.js** die preprocess-Option ergänzen:

```
import adapter from '@sveltejs/adapter-auto';
import { vitePreprocess } from '@sveltejs/vite-plugin-svelte';

/** @type {import('@sveltejs/kit').Config} */
const config = {
  kit: {
    adapter: adapter(),
  },
  preprocess: vitePreprocess(),
};

export default config;
```

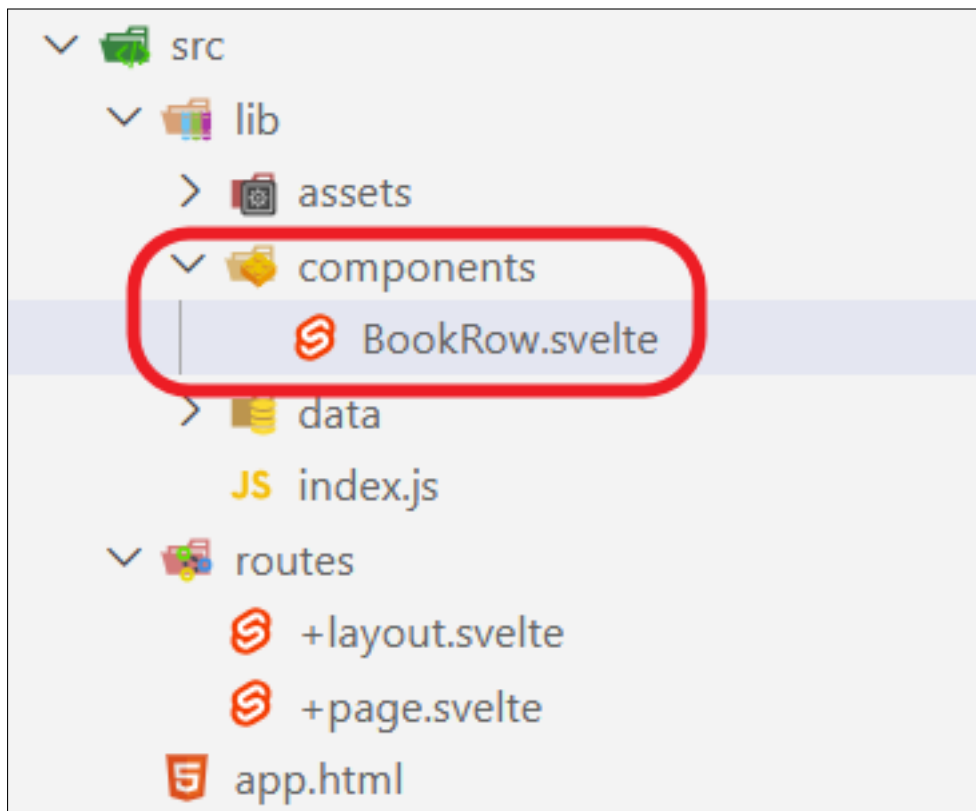
Nun möchten wir gerne die Darstellung einer **Zeile** in eine eigene **Komponente** auslagern.

Komponente erstellen

Eine Komponente ist eine eigenständige Svelte-Datei, die ein bestimmtes UI-Element oder eine Funktionalität kapselt. In unserem Fall möchten wir eine Komponente für die Anzeige einer einzelnen Buchzeile erstellen. Sie ist eine einfache Datei mit der Endung `.svelte`, z.B. **BookRow.svelte**.

Theoretisch könnte man eine Komponente an jeder beliebigen Stelle im Projekt erstellen, aber es ist üblich, sie in einem eigenen Ordner zu organisieren, z.B. **src/lib/components/**.

Erstellen wir also dieses Verzeichnis und legen dort eine neue Datei namens **BookRow.svelte** an:



Wie wir es schon bei der `+page.svelte`-Datei gesehen haben, besteht eine Komponente aus drei Hauptteilen: dem Skript, dem Markup und den Styles.

Wir verlagern jetzt den Code (Skripte, Markup und Styles), welche nur für die Darstellung einer

Zeile erforderlich sind, aus der `+page.svelte`-Datei in die `BookRow.svelte`-Datei. Das Skript könnte wie folgt aussehen:

```
<script>
  let book;
</script>

<div class="row">
  <div class="cell id">{book.id}</div>
  <div class="cell title">{book.title}</div>
  <div class="cell author">{book.author}</div>
  <div class="cell year">{book.year}</div>
  <div class="cell actions">
    <!-- svelte-ignore ally_consider_explicit_label -->
    <button class="mdl2 warn">
      <i class="bi bi-trash"></i>
    </button>
  </div>
</div>

<style lang="scss">
  <!-- siehe style.scss -->
</style>
```

WICHTIG: Man muss sich bei der Vergabe von Style-Klassen in einer Komponente keine Sorgen machen, dass es zu Konflikten mit anderen Komponenten kommt. Svelte sorgt automatisch dafür, dass die Styles einer Komponente nur auf diese Komponente angewendet werden. Das bedeutet, dass die Klasse `.row` in `BookRow.svelte` keine Auswirkungen auf andere `.row`-Klassen in anderen Komponenten hat.

Daten an die Komponente übergeben

Damit die BookRow-Komponente weiß, welches Buch sie anzeigen soll, müssen wir ihr die entsprechenden Daten übergeben. In Svelte geschieht dies über die **\$props()**-Rune. Der Name der Rune (= Variable) wird zum Namen der Eigenschaft (= Attribut), die wir übergeben möchten. In unserem Fall könnte das so aussehen:

```
<script>
  let { book } = $props();
</script>
```

HINWEIS: Warum verwendet man hier die Destrukturierung? Weil es übersichtlicher ist, als `let book = $props().book;` zu schreiben. Es ist eine gängige Praxis in Svelte, die Destrukturierung zu verwenden, um die benötigten Props direkt zu extrahieren.

Standardwerte

Man kann auch Standardwerte für Props festlegen, falls sie nicht übergeben werden. Das könnte so aussehen:

```
<script>
  let { book = { id: 0, title: 'Unknown', author: 'Unknown', year:
    → 'Unknown' } } = $props();
</script>
```

Verwendung in +page.svelte

Nachdem wir die BookRow-Komponente erstellt und die Prop definiert haben, können wir sie in der `+page.svelte`-Datei verwenden. Dazu müssen wir sie zuerst importieren:

```
<script>
  import books from '$lib/data/books.json';
```



```
import BookRow from '$lib/components/BookRow.svelte';

let bookList = $state.raw(books);

function deleteBook(book) {
  console.log(`deleteBook(...) - book.id = ${book.id}`);
  bookList = bookList.filter((x) => x.id !== book.id);
}
</script>
```

Und dann können wir die BookRow-Komponente innerhalb der #each-Schleife verwenden, um jede Buchzeile anzuzeigen:

```
<div class="table">
  {#each bookList as book}
    <BookRow {book} />
  {/each}
</div>
```

In diesem Beispiel übergeben wir die book-Daten als Prop an die BookRow-Komponente. Die Syntax {book} ist eine Kurzform für book={book}, was bedeutet, dass die Variable book als Prop mit dem gleichen Namen an die Komponente übergeben wird.

Output-Events

In unserem ursprünglichen Code hatten wir eine deleteBook-Funktion, die aufgerufen wurde, wenn der Benutzer auf den Lösch-Button klickte. Da sich dieser Button jetzt in der BookRow-Komponente befindet, müssen wir einen Weg finden, um das Löschen eines Buches von der BookRow-Komponente aus zu ermöglichen.

In Svelte ist der Output einer Komponente eine normale Property der \$props()-Rune.

Das bedeutet, dass wir eine Funktion als Prop übergeben können, die von der BookRow-Komponente aufgerufen wird, wenn der Benutzer auf den Lösch-Button klickt.

Fügen wir also in der BookRow.svelte-Datei eine neue Prop namens onDelete hinzu:

```
<script>
  let { book, onDelete } = $props();
</script>
<div class="row">
  <div class="cell id">{book.id}</div>
  <div class="cell title">{book.title}</div>
  <div class="cell author">{book.author}</div>
  <div class="cell year">{book.year}</div>
  <div class="cell actions">
    <!-- svelte-ignore ally_consider_explicit_label -->
    <button onclick={() => onDelete(book)} class="mdl2 warn">
      <i class="bi bi-trash"></i>
    </button>
  </div>
</div>
```

In der aufrufenden Komponente (+page.svelte) müssen wir nun die deleteBook-Funktion als Prop an die BookRow-Komponente übergeben:

```
<div class="table">
  {#each bookList as book}
    <BookRow {book} onDelete={deleteBook} />
  {/each}
</div>
```

Jetzt wird die deleteBook-Funktion in der +page.svelte-Datei aufgerufen, wenn der Benutzer auf den Lösch-Button in der BookRow-Komponente klickt. Dadurch können wir die Logik zum Löschen eines Buches in der übergeordneten Komponente behalten, während die

BookRow-Komponente nur für die Darstellung zuständig ist.

HINWEIS: Die Schreibweise `onDelete={deleteBook}` ist eine Kurzform für `onDelete={x => deleteBook(x)}`, da die `deleteBook`-Funktion bereits die richtige Signatur hat, um direkt als Callback verwendet zu werden. Svelte erkennt automatisch, dass `onDelete` eine Funktion ist und ruft sie mit den entsprechenden Argumenten auf, wenn der Button geklickt wird.

Event-Forwarding

Wenn man in einer Komponente ein Event eines Elements nur weiterleiten will, gibt man den **eventName** ohne Eventhandler an. Das könnte so aussehen:

```
<div onmouseover={overBook} {onmouseleave}>
  ...
</div>
```

In der verwendenden Komponente könnte man dann so darauf reagieren:

```
<BookRow {book} onmouseover={_ => isHighlight = true} onmouseleave={_
  => isHighlight = false} />
```

In diesem Beispiel wird das `mouseover`-Event von der `BookRow`-Komponente weitergeleitet, und die übergeordnete Komponente kann darauf reagieren, indem sie den `isHighlight`-Status entsprechend ändert. Das gleiche gilt für das `mouseleave`-Event.

Two-Way-Binding

Das Binden von Daten zwischen der Komponente und der übergeordneten Komponente wird als **Two-Way-Binding** bezeichnet. Wenn sich die Daten in der `BookRow`-Komponente ändern, sollen diese Änderungen automatisch in der übergeordneten Komponente reflektiert werden, und umgekehrt.

Angenommen, wir möchten die Möglichkeit haben, den Titel eines Buches direkt in der BookRow-Komponente zu bearbeiten. Wir könnten ein Eingabefeld hinzufügen, das den Titel anzeigt und bearbeitbar macht:

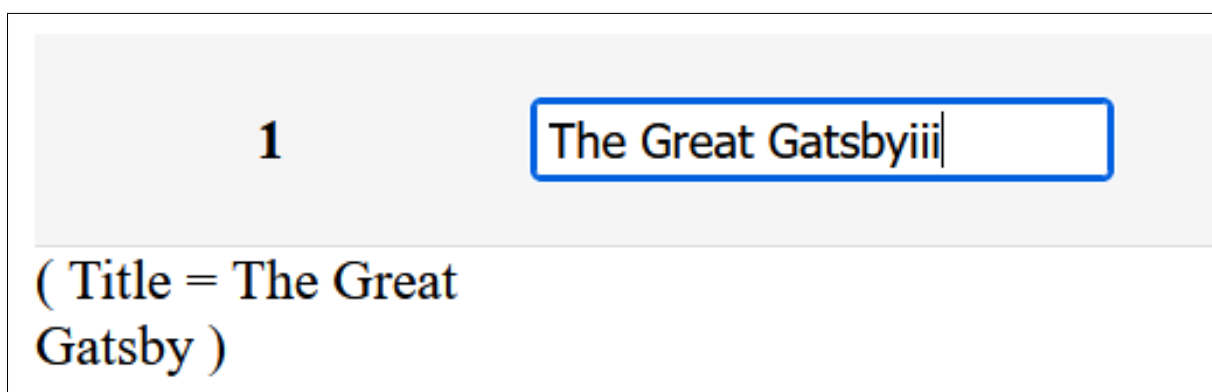
```
<div class="cell title">
  <input type="text" bind:value={book.title} />
</div>
```

In diesem Beispiel verwenden wir die `bind:value`-Syntax, um den Wert des Eingabefelds an die `book.title`-Eigenschaft zu binden. Wenn der Benutzer den Titel ändert, soll die Änderung automatisch in der übergeordneten Komponente reflektiert werden, da `book` eine Referenz auf das Buchobjekt ist, das in der übergeordneten Komponente verwaltet wird.

Zeigen wir zum Testen den Titel eines Buches in der `+page.svelte`-Datei an, um zu sehen, ob die Änderungen korrekt reflektiert werden:

```
<div class="table">
  {#each bookList as book}
    <BookRow {book} onDelete={deleteBook} />
    ( Title = {book.title} )
  {/each}
</div>
```

Wie man sieht, funktioniert es leider noch nicht:



Man muss explizit mit `$bindable()` den Datenfluss vom Child zum Parent aktivieren, indem man die gewünschte Variable mit `$bindable()` zuweist:

```
<script>
  let { book = $bindable(), onDelete } = $props();
</script>
```

Außerdem müssen wir in der `+page.svelte`-Datei die `bookList`-Variable mit `$state()` zuweisen, damit die Änderungen an den Büchern korrekt verfolgt werden und wir müssen ein Binding definieren. Beachte, dass wir hier mit dem Index arbeiten, um die richtige Position in der `bookList` zu aktualisieren, da wir sonst eine Fehlermeldung bekommen:

```
<script>
  ...
  let bookList = $state(books);
  ...
</script>

<h1>Books</h1>
<hr />
Anzahl Bücher: {bookList.length}
<div class="table">
  {#each bookList as book, index (book.id)}
    <BookRow bind:book={bookList[index]} onDelete={deleteBook} />
    ( Title = {book.title} )
  {/each}
</div>
```

Jetzt funktioniert das Two-Way-Binding wie erwartet. Wir können z.B. in der Parent-Komponente einen Button zum Hinzufügen eines neuen Buches erstellen, und die Änderungen werden automatisch in der Child-Komponente reflektiert:

```
<h1>Books</h1>
```

```
<hr />
Anzahl Bücher: {bookList.length}
<div class="table">
  {#each bookList as book, index (book.id)}
    <BookRow bind:book={bookList[index]} onDelete={deleteBook} />
    ( Title = {book.title} )
  {/each}
</div>
<hr />
<button
  onclick={() =>
    (bookList = [
      ...bookList,
      { id: Date.now(), title: 'Neues Buch', author: 'Unbekannt',
→   year: 2024 },
    ])}
>
  <i class="bi bi-plus"></i> Buch hinzufügen
</button>
```

Input-Änderungen

Die Input-Parameter werden zu Beginn an die Sub-Komponente übergeben. Möchte man in der Komponente darauf reagieren, wenn sich dieser Input-Wert in der Master-Komponente ändert, funktioniert das so, wie bei einer lokalen Variable auch: man registriert einen Handler mit `$derived{...}`.

Wir können z.B. in der `BookRow.svelte`-Datei einen Handler hinzufügen, der auf Änderungen am `book`-Objekt reagiert:

```
<script>
```

```
let { book = $bindable(), onDelete } = $props();

$derived(() => {
  console.log('BookRow - book changed:', book);
});
</script>
```

Übungsaufgabe

Finalisiere die Bücherverwaltungs-App, indem du die folgenden Funktionen implementierst:

1. Es soll ein **neues Buch** hinzugefügt werden können. Der Bereich soll nur eingeblendet werden, wenn man auf einen Button klickt. Danach soll man die Details des neuen Buches eingeben können. Nur wenn alle Felder ausgefüllt sind, soll ein Button "Speichern" aktiviert werden (sonst ist er deaktiviert). Nach dem Speichern soll das neue Buch in der Liste angezeigt werden. Ein "Abbrechen"-Button soll die Eingabe zurücksetzen und den Eingabebereich ausblenden.
2. Es soll möglich sein, die Details eines bestehenden Buches zu **bearbeiten**. Dazu soll ein "Bearbeiten"-Button in der BookRow-Komponente hinzugefügt werden, der die Details des Buches in Eingabefelder umwandelt. Nach dem Bearbeiten soll ein "Speichern"-Button die Änderungen übernehmen, und ein "Abbrechen"-Button soll die Änderungen verwerfen und die ursprünglichen Details wiederherstellen.
3. Es soll eine **Bestätigungsabfrage** implementiert werden, bevor ein Buch gelöscht wird. Wenn der Benutzer auf den Lösch-Button klickt, soll ein Dialog angezeigt werden, der fragt: "Möchten Sie dieses Buch wirklich löschen?" mit den Optionen "Ja" und "Nein". Nur wenn der Benutzer "Ja" auswählt, soll das Buch aus der Liste entfernt werden.
4. Es soll eine **Suchfunktion** hinzugefügt werden, mit der die Benutzer die Bücherliste nach Titel, Autor oder Jahr filtern können. Ein Suchfeld soll über der Bücherliste platziert werden, und die Liste soll dynamisch aktualisiert werden, während der Benutzer tippt.